

第七模块：系统稳定性与异常恢复机制（内核级超详细版 • Cortex-M 装备方向）

本模块目标：让系统在真实环境中“不会死”。

重点不是简单介绍看门狗，而是： - HardFault 底层定位机制 - 异常栈帧解析 - 看门狗架构设计 - 自动恢复系统（Self-Recovery Architecture）

本模块属于真正工程分水岭内容。

一、为什么嵌入式系统最终都会遇到异常

长期运行设备不可避免会遇到：

- 指针错误
- 栈溢出
- DMA竞争
- 总线错误
- Flash读写冲突

工程认知：

👉 稳定系统不是“没有错误”，而是“能恢复”。

二、HardFault 的底层机制（必须真正理解）

2.1 HardFault 不是一个错误，而是异常升级

Cortex-M 异常等级：

- UsageFault
- BusFault
- MemManageFault

如果这些异常未开启或未处理：

➡ 会升级为 HardFault。

2.2 HardFault 时 CPU 自动压栈内容

```
R0  
R1  
R2  
R3  
R12  
LR  
PC  
xPSR
```

关键点：

PC 就是出错指令地址。

2.3 如何从栈帧恢复现场（核心能力）

HardFault_Handler 中常见做法：

- 判断使用 MSP 还是 PSP
- 读取栈中 PC
- 反查源码位置

这是定位问题最快的方法。

三、常见 HardFault 根因（装备系统高频）

3.1 未对齐访问

例如：

```
uint32_t* p = (uint32_t*)0x20000001;
```

3.2 执行非法地址

典型原因：

- 函数指针被破坏
 - 栈溢出覆盖返回地址
-

3.3 DMA 写坏内存

当 DMA 地址配置错误：

可能直接破坏栈或代码区。

四、异常恢复架构（Self-Recovery Design）

4.1 为什么不能只打印错误

很多系统 HardFault 后：

- 死循环
- 打日志

但工业系统要求：

👉 自动恢复。

4.2 推荐恢复流程

1) 记录异常信息 (PC/LR) 2) 写入 Flash 日志区 3) 安全重启

4.3 异常计数器（防止重启风暴）

设计：

- 每次异常计数+ 1
 - 若连续异常超过阈值 ➡ 进入安全模式 (Safe Mode)
-

五、看门狗架构设计（不是简单喂狗）

5.1 错误的喂狗方式

```
while(1){
    feed_dog();
}
```

问题：

即使系统部分死锁，仍会喂狗成功。

5.2 正确的健康检查模型 (Health Monitor)

设计一个监控任务：

- 各模块上报“心跳”
- 全部正常才喂狗

模块示例：

- 控制任务
 - 通信任务
 - 日志任务
-

5.3 IWDG 与 WWDG 区别 (底层视角)

IWDG：

- 独立时钟
- 更可靠

WWDG：

- 窗口机制
 - 可检测跑飞
-

六、异常日志系统 (黑匣子设计)

6.1 为什么必须有异常日志

很多问题现场无法复现。

需要记录：

- PC
 - LR
 - Fault Status Register
-

6.2 环形日志区设计

Flash中保留一块区域：

- 记录最近N次异常
 - 覆盖最旧记录
-

七、Fault Status Register（真正底层）

关键寄存器：

- CFSR
- HFSR
- MMFAR
- BFAR

作用：

指示具体异常类型与地址。

八、高频底层考点（标准答案）

Q 1：HardFault 时如何定位问题？

标准答案：

读取异常栈帧中的 PC 与 Fault Status Register。

Q 2：为什么不能无条件喂狗？

标准答案：

系统部分失效仍可能喂狗成功，无法真正检测故障。

Q 3：为什么要设计安全模式？

标准答案：

防止连续异常导致设备反复重启无法恢复。

Q 4：异常恢复的核心是什么？

标准答案：

记录状态 → 安全重启 → 可回溯分析。

九、工程注意事项（装备级经验）

- 1) HardFault Handler 中不要使用复杂函数。
 - 2) 异常日志写入必须保证原子性。
 - 3) 看门狗时间窗口要大于最坏执行时间。
 - 4) 安全模式必须最小化功能。
 - 5) 统计异常频率用于远程维护。
-

十、深度练习（强烈建议）

- 故意制造未对齐访问并解析CFSR。
 - 写一个 HardFault 解析函数打印 PC。
 - 设计一个健康监控喂狗系统。
-

完成本模块后，你将具备“异常可恢复系统设计能力”，这是装备类嵌入式最核心的稳定性能力。